

PRACTICAL : WEEK 6

Task - 1:

Create a client-server application using Named Pipes (FIFOs). The client sends messages and the server processes and responds to them. Analyze FIFO behavior when multiple clients communicate simultaneously.

CODE:

Server.c

```
GNU nano 8.7.1
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/stat.h>

#define REQUEST_FIFO "/tmp/request_fifo"
#define RESPONSE_FIFO "/tmp/response_fifo"

int main()
{
    int request_fd, response_fd;
    char message[100];
    char response[150];

    // Create FIFOs
    mkfifo(REQUEST_FIFO, 0666);
    mkfifo(RESPONSE_FIFO, 0666);

    printf("Server started...\n");
    printf("Waiting for client message...\n");

    // Open request FIFO for reading
    request_fd = open(REQUEST_FIFO, O_RDONLY);

    // Open response FIFO for writing
    response_fd = open(RESPONSE_FIFO, O_WRONLY);

    while (1)
    {
        memset(message, 0, sizeof(message));

        if (read(request_fd, message, sizeof(message)) > 0)
        {
            printf("Client: %s", message);

            if (strncmp(message, "exit", 4) == 0)
            {
                strcpy(response, "Server shutting down...\n");
                write(response_fd, response, strlen(response));
                break;
            }

            // Process the client's message
            snprintf(response, sizeof(response),
                "Server processed: %s", message);

            write(response_fd, response, strlen(response));

            printf("Response sent to client.\n");
            printf("Waiting for client message...\n");
        }
    }

    close(request_fd);
    close(response_fd);

    unlink(REQUEST_FIFO);
    unlink(RESPONSE_FIFO);

    return 0;
}
```

Client.c

```
GNU nano 8.7.1
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <fcntl.h>

#define REQUEST_FIFO "/tmp/request_fifo"
#define RESPONSE_FIFO "/tmp/response_fifo"

int main()
{
    int request_fd, response_fd;
    char message[100];
    char response[150];

    // Open request FIFO for writing
    request_fd = open(REQUEST_FIFO, O_WRONLY);

    // Open response FIFO for reading
    response_fd = open(RESPONSE_FIFO, O_RDONLY);

    printf("Client connected to server.\n");

    while (1)
    {
        printf("Enter message: ");
        fgets(message, sizeof(message), stdin);

        // Send message to server
        write(request_fd, message, strlen(message));

        // Receive response from server
        memset(response, 0, sizeof(response));
        read(response_fd, response, sizeof(response));

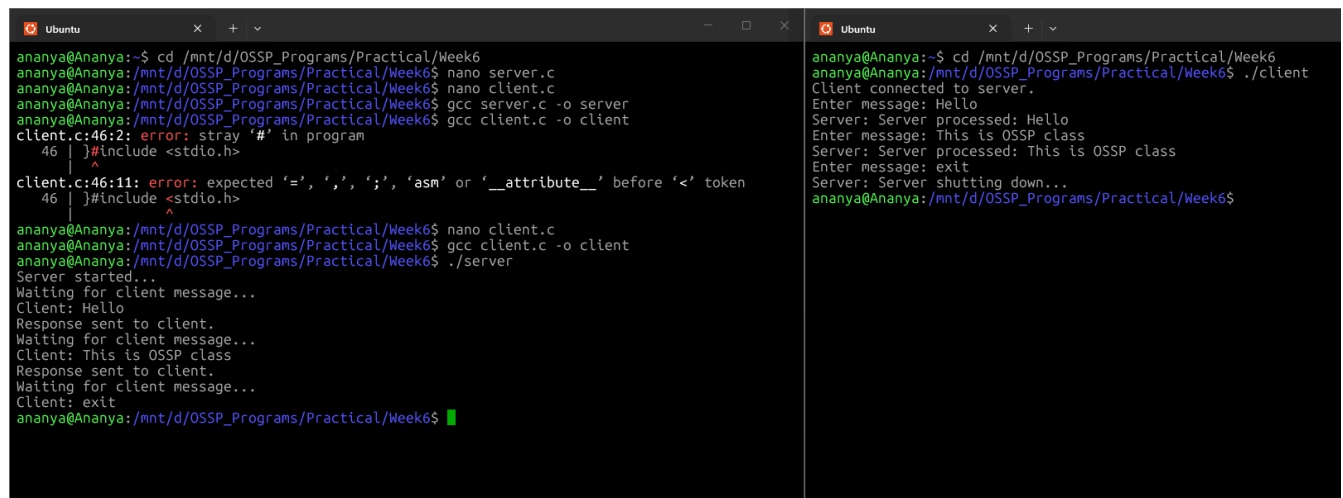
        printf("Server: %s", response);

        if (strncmp(message, "exit", 4) == 0)
            break;
    }

    close(request_fd);
    close(response_fd);

    return 0;
}
```

OUTPUT:



```
ananya@Ananya:~$ cd /mnt/d/OSSP_Programs/Practical/Week6
ananya@Ananya:/mnt/d/OSSP_Programs/Practical/Week6$ nano server.c
ananya@Ananya:/mnt/d/OSSP_Programs/Practical/Week6$ nano client.c
ananya@Ananya:/mnt/d/OSSP_Programs/Practical/Week6$ gcc server.c -o server
ananya@Ananya:/mnt/d/OSSP_Programs/Practical/Week6$ gcc client.c -o client
client.c:46:2: error: stray '#' in program
  46 | }#include <stdio.h>
      | ^
client.c:46:11: error: expected '=', ',', ';', 'asm' or '__attribute__' before '<' token
  46 | }#include <stdio.h>
      | ^
ananya@Ananya:/mnt/d/OSSP_Programs/Practical/Week6$ nano client.c
ananya@Ananya:/mnt/d/OSSP_Programs/Practical/Week6$ gcc client.c -o client
ananya@Ananya:/mnt/d/OSSP_Programs/Practical/Week6$ ./server
Server started...
Waiting for client message...
Client: Hello
Response sent to client.
Waiting for client message...
Client: This is OSSP class
Response sent to client.
Waiting for client message...
Client: exit
ananya@Ananya:/mnt/d/OSSP_Programs/Practical/Week6$

ananya@Ananya:~$ cd /mnt/d/OSSP_Programs/Practical/Week6
ananya@Ananya:/mnt/d/OSSP_Programs/Practical/Week6$ ./client
Client connected to server.
Enter message: Hello
Server: Server processed: Hello
Enter message: This is OSSP class
Server: Server processed: This is OSSP class
Enter message: exit
Server: Server shutting down...
ananya@Ananya:/mnt/d/OSSP_Programs/Practical/Week6$
```

OBSERVATION:

The client-server communication system was implemented successfully using Named Pipes (FIFOs). The client process sends messages through the client FIFO, and the server process reads, processes, and sends a response through the server FIFO. The experiment demonstrated two-way communication between the client and server processes using named pipes. The FIFO maintains the data in First-In-First-Out order, and multiple clients can communicate with the same server through the shared FIFO. This experiment demonstrates the use of named pipes for inter-process communication between separate processes.

Task - 2:

Create a POSIX signal handling program that captures SIGINT, SIGTERM, and SIGUSR1. Demonstrate asynchronous event handling and explain the role of signal handlers.

CODE:

```
GNU nano 8.7.1 signal.c
#include <stdio.h>
#include <unistd.h>
#include <signal.h>

void signal_handler(int signal)
{
    if (signal == SIGINT)
        printf("\nSIGINT received.\n");

    else if (signal == SIGTERM)
        printf("SIGTERM received.\n");

    else if (signal == SIGUSR1)
        printf("SIGUSR1 received.\n");
}

int main()
{
    signal(SIGINT, signal_handler);
    signal(SIGTERM, signal_handler);
    signal(SIGUSR1, signal_handler);

    printf("Program is running.\n");
    printf("PID: %d\n", getpid());

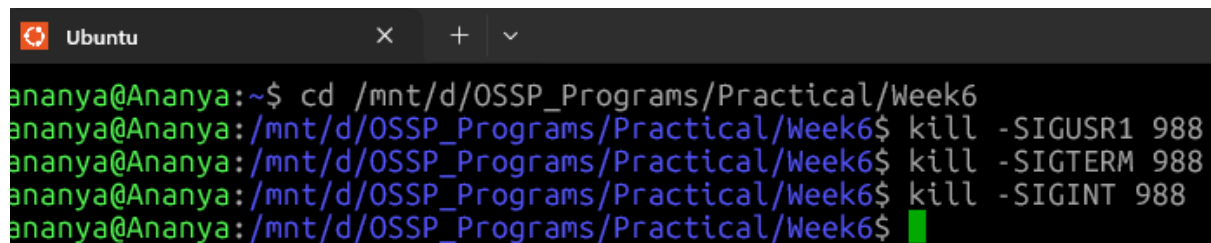
    while (1)
    {
        printf("Waiting for signals...\n");
        sleep(5);
    }

    return 0;
}
```

OUTPUT:

```
ananya@Ananya:/mnt/d/OSSP_Programs/Practical/Week6$ nano signal.c
ananya@Ananya:/mnt/d/OSSP_Programs/Practical/Week6$ gcc signal.c -o signal
ananya@Ananya:/mnt/d/OSSP_Programs/Practical/Week6$ ./signal
Program is running.
PID: 988
Waiting for signals...
Waiting for signals...
Waiting for signals...
Waiting for signals...
Waiting for signals...
Waiting for signals...
Waiting for signals...
Waiting for signals...
Waiting for signals...
SIGUSR1 received.
Waiting for signals...
Waiting for signals...
Waiting for signals...
SIGTERM received.
Waiting for signals...
Waiting for signals...

SIGINT received.
Waiting for signals...
```



```
ananya@Ananya:~$ cd /mnt/d/OSSP_Programs/Practical/Week6
ananya@Ananya:/mnt/d/OSSP_Programs/Practical/Week6$ kill -SIGUSR1 988
ananya@Ananya:/mnt/d/OSSP_Programs/Practical/Week6$ kill -SIGTERM 988
ananya@Ananya:/mnt/d/OSSP_Programs/Practical/Week6$ kill -SIGINT 988
ananya@Ananya:/mnt/d/OSSP_Programs/Practical/Week6$ █
```

OBSERVATION:

The program successfully captured and handled the SIGINT, SIGTERM, and SIGUSR1 signals using POSIX signal handling. A signal handler was registered for each signal, and when a signal was sent to the running process, the corresponding handler was executed and the appropriate message was displayed. The experiment demonstrated asynchronous event handling, where signals can be received by a process at any time during its execution. Thus, the role of signal handlers in responding to and managing asynchronous events was successfully demonstrated.